

How the internal queue works, what decisions the scheduler makes, and what knowledge it requires

A detailed, readable guide to the single-scheduler / single-worker path, with concrete examples and implementation implications for a harness-aware controller.

The scheduler repeatedly answers two different questions: Which waiting request should be considered first? And which of those requests can safely fit on the GPU right now?

Document	Details
Scope	One scheduler and one worker group; no multi-worker routing or prefill/decode disaggregation
Code basis	Current SGLang main branch inspected September 14, 2026
Core topics	Queue states, ranking policies, prefill admission, decode, KV pressure, priority, request metadata, and controller hooks
Primary sources	Official SGLang source code, server-argument documentation, and the SGLang paper

Version note: SGLang changes quickly. The concepts in this guide are stable, but defaults, flags, aliases, and exact source-line locations can change between releases.

Prepared for implementation planning with Codex

Contents

1. Executive mental model
2. The scheduler is not one simple queue
3. Request arrival and internal representation
4. The repeating scheduling loop
5. Queue ranking policies
6. Resource admission and memory estimation
7. Long prompts and chunked prefill
8. Prefill and decode interaction
9. Decode lifecycle, memory pressure, and retraction
10. Priority preemption
11. Complete knowledge and dependency inventory
12. What the scheduler does not know
13. Concrete resumed-agent scheduling example
14. The three-machine model
15. Implications for a harness-aware controller
16. Decision pseudocode and implementation checklist
17. Primary sources

Executive answer

Yes. The SGLang scheduler requires knowledge of incoming requests. It cannot schedule opaque requests blindly. It needs prompt token structure, length, output bounds, cache matches, priority and compatibility metadata. It combines that request knowledge with live worker state such as free KV blocks, evictable cache, request slots, currently decoding requests, and configured token budgets.

The scheduler does not normally understand the business meaning of a request. It does not infer that a prompt belongs to a customer-support agent, that a user is waiting after a tool call, or that the request must produce its first token within 100 ms. Those facts must be translated by a harness or controller into scheduler-visible metadata and actions.

The most important distinction is between ranking and admission:

- Ranking decides the order in which waiting requests are considered.
- Admission decides whether a considered request can fit safely and legally in the next GPU batch.
- Priority can improve ranking and may permit preemption, but it cannot bypass KV capacity, grammar readiness, LoRA compatibility, request-slot limits, or a GPU kernel already in flight.

1. Executive mental model

For ordinary autoregressive inference, a request has two main computational phases:

Phase	What happens	Typical resource shape
Prefill / extend	The model processes the prompt tokens that are not already represented in reusable KV cache.	Often compute-heavy; work scales with uncached prompt length.
Decode	The model generates output tokens incrementally using the request's accumulated KV state.	Often memory-bandwidth-sensitive; KV grows as tokens are generated.

The scheduler must decide both who runs and what kind of GPU pass runs next. It continually forms CPU-side `ScheduleBatch` objects. The model runner converts these into lower-level `ForwardBatch` objects whose data is primarily GPU tensors.

Scheduling is not merely queue management. It is a joint decision across request order, prompt computation, KV capacity, cache reuse, compatibility constraints, and request lifecycle.

2. The scheduler is not one simple queue

For the one-worker path, the important runtime states are:

State	Type / role	Meaning
grammar_queue	Asynchronous readiness queue	Requests waiting for JSON, regex, or other grammar compilation. They are not yet eligible for normal prefill admission.
waiting_queue	Python list of Req objects	Requests ready to be ranked and considered for prefill. The list can be reordered on every scheduling pass.
chunked_req	Single parked partial-prefill request	A long prompt whose prefill has started but whose remaining prompt tokens must be processed in later chunks.
running_batch	<code>ScheduleBatch</code>	The continuously changing batch of requests that have completed prefill and are currently decoding.
last_batch	Previous <code>ScheduleBatch</code>	The previously launched batch, used to merge prefill results and support CPU/GPU overlap.
result_queue	Overlap state	Pending batch results used when CPU scheduling is pipelined with device execution.

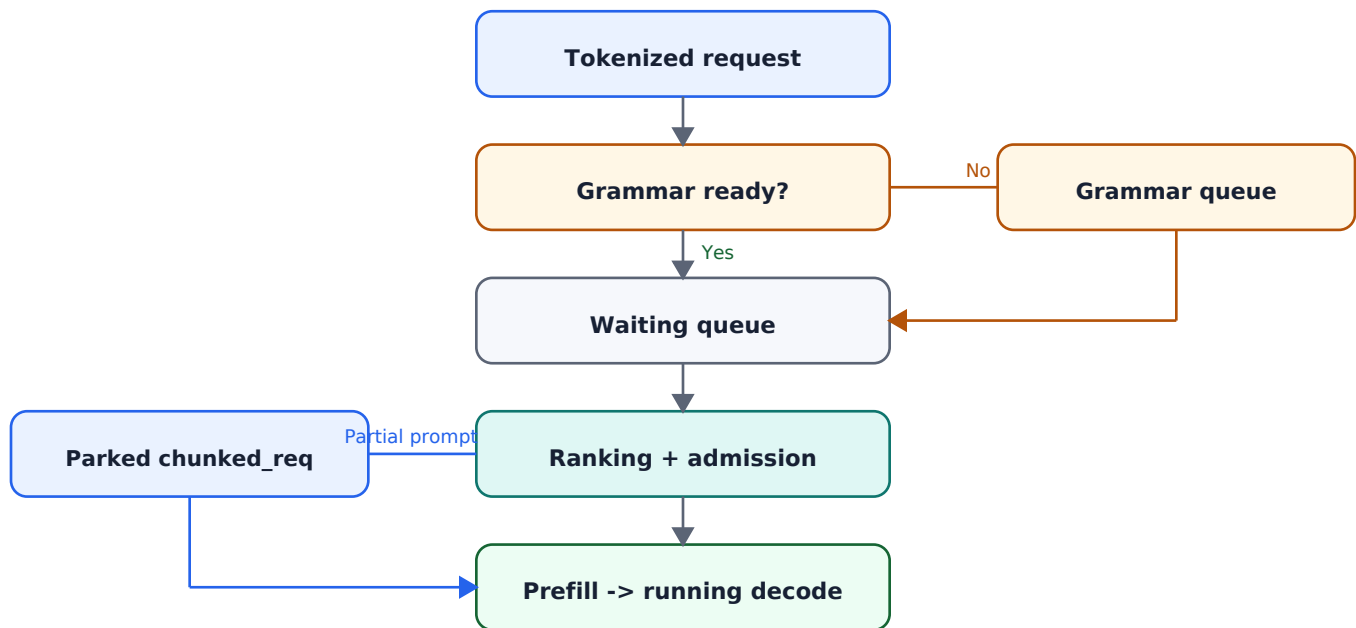


Figure 1. Simplified request lifecycle for the ordinary one-worker path.

A request can move backward. If a decoding request is retracted because of KV pressure, or preempted for a sufficiently higher-priority request, it can return to the waiting queue.

3. Request arrival and internal representation

The scheduler usually does not receive raw frontend text directly. The API/tokenizer layer tokenizes and normalizes the request first. The scheduler receives a structured `TokenizedGenerateReqInput`, constructs an internal `Req`, validates it, and then decides which readiness queue it belongs in.

The internal request can contain:

Category	Representative request knowledge
Identity	Request ID, session ID and timestamps
Prompt structure	Original input text, prompt token IDs and any input embeddings
Generation bound	Sampling parameters, especially <code>max_new_tokens</code> , stop behavior and EOS handling
Importance	Optional integer priority
Cache namespace	<code>extra_key</code> , <code>cache_salt</code> , session generation and reusable-prefix identity
Scheduling locality	<code>routing_key</code>
Model variation	LoRA adapter ID
Output constraints	Grammar object or grammar compilation request
Execution features	Streaming, logprobs, hidden states, embeddings, beam width and multimodal data

Category	Representative request knowledge
Lifecycle	Output tokens produced so far, cached-prefix indices, chunk/retraction state and finish reason

Before ordinary queue insertion, SGLang performs the following steps:

- Validate the request and the supported feature combination.
- Initialize or normalize the maximum number of output tokens.
- Validate the prompt against the model's context and request-input limits.
- Expand or validate multimodal inputs when present.
- Send grammar-constrained requests to grammar compilation if the grammar is not ready.
- Otherwise, validate or assign priority, check the maximum queue limit, initiate storage-level KV prefetch when configured, append to `waiting_queue`, and record queue-entry/aging timestamps.

The scheduler sees token IDs and explicit metadata. It does not need to understand the natural-language meaning of those tokens to perform ordinary scheduling.

4. The repeating scheduling loop

At a high level, every scheduler iteration performs:

1. Receive newly arrived generation requests and control messages.
2. Process or reconcile the result of the previous GPU batch.
3. Merge requests that completed prefill into the running decode batch.
4. Remove requests that have finished or aborted.
5. Check whether grammar compilation or hierarchical-cache events made additional requests ready.
6. Try to construct a new prefill batch from the waiting queue and any active chunked request.
7. If a prefill batch can be built, select it. Otherwise, update and select the running decode batch.
8. Launch the selected batch, process outputs, stream eligible results, update statistics, and repeat.

In overlap mode, CPU-side scheduling and metadata work can be pipelined with GPU execution. That is why the scheduler tracks both the current running batch and the previously launched batch. The scheduling decision is still made at batch boundaries; overlap does not provide arbitrary mid-kernel interruption.

5. Queue ranking policies

Immediately before prefill admission, SGLang recalculates the ordering of `waiting_queue`. The method is called `calc_priority()`, but 'priority' here means the active scheduling policy's ordering, not necessarily business priority.

Policy	Knowledge consumed	Ordering behavior
FCFS	Queue-entry time	Oldest request first.
Priority + FCFS	Priority plus queue-entry time	Higher priority first by default; ties use arrival order. A flag can reverse the numeric direction.
LPM	Token-level prefix match against the radix cache	Longest reusable prefix first.
DFS-weight	Location of matched prefixes in the radix tree	Groups requests under shared cache subtrees.
HRRN	Uncached length and token-based waiting age	Balances efficiency with aging to reduce starvation.
LOF	<code>max_new_tokens</code>	Longest declared output first; priority can be used as the primary key in the current implementation.
Random	No request estimate	Shuffles the waiting list.
Routing-key	Request <code>routing_key</code> and key frequency in <code>running_batch</code>	Prefers keys already common among running requests.

The current documented default is `fcfs`. Documentation may also expose a priority policy alias, while the source's concrete priority sort is expressed through the `priority-scheduling enable` flag combined with `FCFS`-style ordering. Treat exact alias behavior as release-specific.

5.1 FCFS

```
A arrived at 10:00:00
B arrived at 10:00:01
C arrived at 10:00:02

Order: A -> B -> C
```

With no other ranking policy, the arrival order is preserved.

5.2 Priority plus FCFS

With native priority scheduling enabled, the FCFS path sorts by priority first and queue-entry time second. Larger integer values are preferred by default; `--schedule-low-priority-values-first` reverses this interpretation.

Request	Priority	Arrival	Resulting order
Coding prefill	1	10:00:00	2
Support resume	10	10:00:02	1
Reporting job	0	09:59:58	3

Priority changes the consideration order. It does not yet guarantee execution. The request must still pass every resource and compatibility check.

5.3 Longest-prefix-match scheduling

LPM asks how many tokens of each request already have reusable KV. It performs token-exact prefix matching against the radix cache using the request's prompt/output token IDs together with its cache namespace, including `extra_key` and `cache_salt`. Hierarchical-cache configurations can distinguish device, host and storage hits.

Example: a prompt containing 70,100 tokens may need only 100 tokens of new prefill if 70,000 prefix tokens are already cached. That request can be far cheaper than a 10,000-token request with no cache hit.

The original SGLang paper describes RadixAttention as an LRU-managed radix tree that automatically matches, inserts and evicts reusable KV prefixes. Cache-aware scheduling then selects requests to improve reuse and avoid redundant prefill computation.

5.4 In-batch prefix opportunity

Current source also contains an in-batch prefix-caching optimization. If multiple waiting requests share a prefix that is not yet substantially cached, the policy may temporarily deprioritize all but one. The first request computes the shared prefix; later requests can then reuse it.

5.5 HRRN aging

A purely cache-first policy can starve requests with poor reuse. HRRN uses token-based aging. Its source-level ratio is approximately:

```
score = 1 +
      (prefill tokens processed since request arrival)
      -----
      (uncached tokens remaining for this request)
```

As the system processes other prompt tokens, a waiting request's score rises. Requests with little uncached work also receive favorable ratios because they are cheap to complete.

5.6 Large-queue fallback

Current source turns off expensive LPM or HRRN prefix ranking when the waiting queue exceeds 128 entries and falls back to FCFS-like processing. Repeatedly matching every large prompt can otherwise make CPU scheduling itself a throughput bottleneck.

6. Resource admission and memory estimation

After ordering the waiting queue, SGLang constructs a `PrefillAdder`. This is the admission controller. It scans waiting requests in policy order and attempts to assemble a batch that fits current limits.

Admission dimension	Information checked
Prefill compute budget	<code>max_prefill_tokens</code> , chunk size, mixed decode-token charge and optional tile budget
Request concurrency	<code>max_running_requests</code> , <code>prefill_max_requests</code> , request-table rows and beam width
KV capacity	Free device KV plus safely evictable cached KV
Existing commitments	Current running decode batch and estimated future token growth

Admission dimension	Information checked
Candidate work	Uncached prompt tokens, host/storage load-back, output reserve and page overhead
Readiness	Grammar compiled, cache prefetch complete and multimodal inputs ready
Compatibility	LoRA combination, logging/embedding features, beam support and backend constraints

6.1 Simplified memory calculation

KV required approximately equals:

```

uncached prompt tokens
+ estimated future output KV
+ page/allocation overhead
+ KV reserved for requests already decoding
+ host/storage load-back charges when applicable

```

The candidate can run only if this fits within:

```

free KV capacity + safely evictable cached KV

```

The actual code includes additional handling for paged allocation, sliding-window attention, Mamba state, beam-search rows, chunk continuation, page alignment and hierarchical cache restoration.

6.2 Why max_new_tokens matters

Consider two requests with the same 2,000 uncached prompt tokens:

Request	Uncached prompt	max_new_tokens	Scheduling implications
A	2,000	50	Small possible decode KV growth
B	2,000	10,000	Much larger possible decode KV growth

The scheduler cannot reserve zero future memory, because every generated token adds KV state. But reserving the full declared maximum for every request would severely reduce concurrency. SGLang therefore estimates future growth using the remaining declared output bound, an internal clip, and a dynamic new_token_ratio.

6.3 The dynamic reservation ratio

Conceptually, a running request's future KV reservation is similar to:

```

future_reserve =
    min(max_new_tokens - tokens_already_generated, internal_clip)
    * new_token_ratio

```

The ratio is a heuristic because SGLang does not know the true future output length. During successful operation it can become more aggressive. After a decode-memory failure and retraction, the scheduler increases reservation conservatism. This is a feedback controller, not an exact runtime predictor.

6.4 Scanning behavior

The scheduler scans the reordered waiting queue. For each candidate it may:

- Skip it temporarily if its LoRA cannot join the active adapter set.
- Skip it if hierarchical-cache prefetch is still in progress.
- Initialize its prefix match and calculate the uncached extension length.
- Admit it fully if all budgets fit.
- Admit a partial chunk if chunked prefill is available and appropriate.
- Attempt priority preemption if the running batch is full and preemption is enabled.
- Stop adding requests when token, request-row, chunk or other batch limits are exhausted.

Admitted requests are removed from `waiting_queue`. Preempted requests are re-added. A newly partial request becomes `chunked_req`. The admitted set is converted into a new `ScheduleBatch`.

7. Long prompts and chunked prefill

Suppose an 80,000-token coding request has little cached KV, while the configured chunk size permits only 8,192 prompt tokens in the next pass.

```

80,000-token prompt
  |
  v
first 8,192-token prefill chunk
  |
  v
park remaining work as chunked_req
  |
  v
process later chunks
  |
  v
complete prefill and join running decode

```

The request does not begin again from zero. KV produced by completed chunks is stashed or cached, and the request resumes from the next unprocessed range. The active `chunked_req` is considered before ordinary waiting requests so its allocated/request state can be handled consistently.

Chunking creates scheduling intervention points between prompt segments. Smaller chunks generally improve responsiveness to newly arriving work but can reduce prefill efficiency through more launches, smaller matrix operations, and additional scheduler overhead. Larger chunks improve throughput but increase the time before urgent work can be reconsidered.

8. Prefill and decode interaction

After cleaning up the previous batch, the scheduler first tries to form a new prefill/extend batch. If it finds one, that batch is selected. If it cannot build one, the scheduler updates and selects the running decode batch. Prefill can be deferred by specific delayer policies or blocked by resource/readiness conditions.

This structure favors time to first token for new requests, while continuous batching keeps already-running requests moving across repeated decode steps.

8.1 Mixed chunked prefill

When enabled and compatible with the request features, SGLang can combine a prompt chunk with one decode token for existing running requests:

Mode	GPU pass 1	GPU pass 2	Effect
Separate	New-request prefill	Existing-request decode	Clear separation, but decode may pause during prefill.
Mixed chunk	Prompt chunk plus decode tokens	Next scheduling turn	Existing decodes can progress while prefill consumes otherwise useful compute.

Some feature combinations, such as specific logprob, embedding, beam, speculative-decoding or multimodal cases, can prevent mixing. The scheduler checks these compatibility constraints before merging batches.

9. Decode lifecycle, memory pressure, and retraction

After a request completes its final prefill chunk:

- Its first model output is processed.
- If it is not finished, it joins `running_batch`.
- Future scheduling iterations include it in decode passes.
- Ordinarily, each decode pass generates the next token for every included request.
- Finished or aborted requests are filtered from the running batch.
- Their KV is released or inserted into the reusable prefix cache according to cache and session policy.

This is continuous batching: the decode batch is not a fixed group created once. Requests join after prefill and leave when finished, aborted, retracted, or preempted.

9.1 Why admission estimates can fail

The scheduler knows `max_new_tokens`, but it does not know exactly when EOS will occur. Many requests stop early; some approach their maximum. The dynamic reservation ratio intentionally trades safety against concurrency, so occasional underestimation is possible.

9.2 Decode-memory recovery

Before preparing another decode step, the scheduler checks whether the batch has sufficient KV memory. If it does not:

- Select one or more running requests for retraction.
- Release enough request-table and KV resources.
- Remove the selected requests from the running batch.
- Return them to the waiting queue.
- Increase the future-memory reservation ratio so later admission is more conservative.
- Abort a request only in severe cases where progress cannot be made safely.

```
Admitted with an optimistic estimate
-> output grows more than expected
-> KV headroom becomes unsafe
-> request is retracted
-> request waits and may need reprocessing
```

Retraction is a recovery mechanism, not a desirable steady-state policy. Frequent retraction wastes work, damages latency, and signals that admission is too aggressive for the workload.

10. Priority preemption

Memory retraction and priority preemption are related but distinct:

Mechanism	Trigger	Purpose
Memory retraction	Decode cannot safely allocate more KV	Recover from insufficient memory and restore forward progress.
Priority preemption	A high-priority waiting request needs capacity held by sufficiently lower-priority work	Reallocate resources toward more important work.

When priority preemption is enabled, the admission loop can attempt to displace lower-priority running requests if the priority difference reaches the configured threshold. Displaced requests return to the waiting queue. The preferred request must still pass normal memory, token-budget, request-slot and compatibility checks after resources are reclaimed.

Priority means: You may reclaim resources from sufficiently less-important work for this request. It does not mean: Ignore memory limits and execute immediately.

Priority also cannot interrupt an arbitrary model kernel halfway through execution. The scheduler acts when control returns at a scheduling/batch boundary. Chunking makes these boundaries more frequent for long prompt work.

11. Complete knowledge and dependency inventory

11.1 Knowledge about each request

Request information	Why the scheduler needs it
Prompt token IDs	Required for exact prefix-cache matching and building model inputs.
Prompt length	Used for context validation and estimating prefill work.
Cached-prefix length	Subtracts reusable work from the prompt and identifies device/host/storage hits.
Uncached extension length	Direct charge against the prefill token and KV allocation budgets.
max_new_tokens	Upper bound used to estimate future decode KV growth; also used by LOF.
Generated tokens so far	Determines remaining output allowance, current sequence length and finish status.
Priority	Used for priority ordering, queue-limit eviction and optional running-request preemption.
Queue-entry time	Used by FCFS and as a tie-breaker under priority.
Arrival processed-token counter	Used by HRRN's token-based aging calculation.
Session ID / generation	Maintains session continuity and cache namespace correctness.
extra_key and cache_salt	Prevent unsafe cache sharing across incompatible tenants, model states or namespaces.
routing_key	Allows routing-key scheduling to favor locality with the running batch.
LoRA ID	Determines whether the adapter is already loaded or can join the next batch.
Grammar readiness	A constrained request cannot enter normal prefill until its grammar object is ready.
Sampling and stop parameters	Determine legal token sampling, completion and output bounds.
Beam width	Consumes multiple request rows and affects admission capacity.
Logprob / hidden-state / embedding needs	Can change batch preparation and restrict mixing with other requests.
Multimodal readiness	Image/video features and padded token representations must be materialized consistently.
Chunked / retracted / finished state	Determines whether to resume, requeue, merge into decode, release resources, or return output.

11.2 Knowledge about the whole worker

Worker/system information	Why it is needed
Free device KV blocks	Whether new prompt/decode state can be allocated now.
Evictable prefix-cache KV	How much additional capacity can be reclaimed safely.
Protected/locked cache state	Which KV cannot be evicted because it is referenced by active work.
Host/storage cache state	Whether required KV is resident, in transfer, missing, or ready for load-back.
Request-table slots	The maximum number of live request rows available to model kernels.
Current running batch	Existing decode occupancy, sequence lengths, priorities, adapters and feature requirements.
Estimated remaining outputs	Future KV reservation for running requests.
Maximum prefill tokens	Upper bound on prompt tokens processed in one prefill batch.
Chunked-prefill size	Maximum partial-prompt work admitted in the current chunk.
Maximum running/prefill requests	Concurrency limits independent of token memory.
Page size / allocation overhead	KV is allocated in pages, so raw token counts must be aligned and include slack.
Attention/backend constraints	Tile budgets, sliding-window pools and model-specific allocation behavior.
Loaded LoRA set	Whether another adapter may join without exceeding capacity or triggering a load/drain action.
Previous batch and pending results	Needed to know which prefills completed, which tokens were produced and which requests finished.
Scheduling configuration	Active policy, numeric priority direction, preemption threshold, delayer settings and overlap/mixing flags.

11.3 Component dependencies

Component	Scheduler dependency
API / tokenizer manager	Produces tokenized structured requests and returns streamed outputs.
Request receiver / dispatcher	Brings requests and control messages into the scheduling process.
Grammar manager	Compiles output constraints and releases ready requests.
SchedulePolicy	Reorders waiting_queue according to FCFS, priority, cache, aging, output or routing policy.
Radix / hierarchical cache	Performs prefix matching, tracks cache ownership, handles eviction and reports transfer readiness.
ReqToTokenPool	Provides request rows that map each active sequence to token/KV locations.
TokenToKVPoolAllocator	Reports capacity and allocates/frees physical KV slots.
PrefillAdder	Combines ranking output with all admission budgets to select feasible requests.
ScheduleBatch	Stores the CPU-side execution plan.
Model runner / worker	Converts the plan to GPU tensors and executes the forward pass.
Output/result processing	Updates request tokens and finish state, streams results, and frees or caches KV.
Metrics and load observers	Track queueing, cache hits, memory utilization, latency and retractions.

12. What the scheduler does not know

The current internal request includes a native integer priority, but it does not natively express the full agentic intent discussed in the controller proposal. In particular, the core request representation does not directly encode:

- A user is actively waiting.
- The request is paused on a tool call.
- The expected tool-completion time and its uncertainty.
- A per-request deadline after tool completion.
- Whether the deadline is hard or soft.
- The business cost of missing that deadline.
- The probability that the session will resume soon.
- An output-length distribution such as P50, P95 or P99 rather than only an upper bound.
- The recomputation cost or business value of preserving a particular KV prefix.
- A preferred cache tier such as keep in host memory but not distant storage.
- A request to reserve decode headroom near a future event.

SGLang has waiting/running timeout mechanisms, but these abort requests after a time limit. That is not the same as earliest-deadline-first scheduling or proactive resource preparation that prevents a deadline miss.

If two requests have the same token shape, cache state, arrival time, priority and compatibility metadata, the scheduler has no business-level reason to distinguish them.

13. Concrete resumed-agent scheduling example

Assume the worker currently has:

- C: Twenty normal requests in `running_batch`, each producing output tokens.
- B: An 80,000-token coding prompt being processed through chunked prefill.
- A: A customer-support agent that has just resumed after a database tool call.

The resumed request arrives with:

```
request A
priority          = 10
complete prompt   = 80,200 tokens
reusable cached prefix= 80,000 tokens
new tool result   = 200 uncached tokens
max_new_tokens    = 400
```

Step 1: queue insertion

SGLang validates priority, checks the queued-request limit, initiates any configured hierarchical-cache prefetch, appends A to `waiting_queue`, and records its queue-entry and aging state.

Step 2: queue reordering

With priority plus FCFS, A moves ahead of normal-priority coding work. Under a cache policy, its large exact cache match would also make it attractive.

Step 3: real prefill work calculation

The full prompt is 80,200 tokens, but the radix cache reports 80,000 reusable tokens. The scheduler therefore plans approximately 200 tokens of new prefill rather than recomputing all 80,200.

Step 4: resource admission

The `PrefillAdder` checks request rows, prefill-token capacity, device KV, future output reserve, host/storage restoration, LoRA compatibility, grammar readiness and backend feature constraints.

Step 5: next GPU batch

If A fits, SGLang constructs a prefill/extend batch containing A. With mixed chunking enabled and compatible features, some or all running decode requests C can contribute their next decode token to the same pass. Coding request B remains parked until another chunk is scheduled.

Step 6: transition to decode

After A's final prefill completes, it joins `running_batch` and begins generating output tokens on later decode iterations.

Reasons the high-priority request can still be delayed

- Its required KV restoration has not completed.
- No request-table row is available.
- Its grammar is not compiled.

- Its uncached prompt plus decode reserve does not fit.
- Priority preemption is disabled or the priority gap is below the threshold.
- No running request can be retracted safely enough to create space.
- The GPU is executing a batch and has not returned to a scheduling boundary.

This is the exact gap between native priority support and a deadline guarantee.

14. The three-machine model

Machine	Question	Inputs	Output
1. Ranking	Which request is most desirable?	Priority, arrival time, prefix match, aging, output bound and routing key.	Reordered waiting_queue.
2. Admission	Which desirable requests can safely fit now?	Uncached tokens, KV capacity, output reserve, request slots, chunk limits, cache-transfer readiness, LoRA and grammar constraints.	Feasible prefill batch, partial chunk, preemption attempt, or no admission.
3. Lifecycle	Should the next pass be prefill, decode or mixed, and what happens afterward?	Admitted work, running batch, previous GPU result, finished state and memory pressure.	ScheduleBatch, request transitions, streamed output and resource release.

The first machine expresses policy. The second protects correctness and memory safety. The third drives continuous batching and request progress.

15. Implications for a harness-aware controller

Current SGLang largely answers:

Given request metadata and current resource state, what feasible batch best matches my configured scheduling policy?

A harness-aware controller would add:

Which request must be ready by when, which agent is likely to resume, and what preparation should occur before that request becomes runnable?

15.1 Three core integration points

Integration point	Controller action	Direct scheduling effect
Ranking input	Translate agent importance, user-waiting state, deadline slack and critical-path status into a native priority or a custom scheduling score.	Changes who is considered first.
Admission / headroom	Temporarily adjust prefill throttling, chunk size, maximum admitted background work, or protected decode capacity near an expected urgent resume.	Changes whether urgent work can fit without waiting for a long background pass.
Cache preparation	Use session/prefix identity and tool-return ETA to retain, demote, restore or prefetch KV in the appropriate tier.	Changes whether the request waits for KV or recomputes its prompt after becoming ready.

15.2 Recommended separation of responsibility

Harness / controller owns	SGLang scheduler owns
Business priority, user-waiting state, deadline, tool ETA, uncertainty, cache value, optional-work classification and admission policy.	Token-exact cache matching, memory-safe batch formation, GPU request rows, physical KV allocation, model compatibility, decode progress and output processing.
Prediction and policy updates when tool timing changes.	Immediate feasibility checks against live worker state.
Choosing what to sacrifice when the system is overloaded.	Executing the selected policy without violating memory or model constraints.

15.3 Important guarantee boundary

No scheduler can guarantee that every deadline is met while also always saturating the GPU and never reserving unused capacity under arbitrary load. Those objectives conflict during overload or prediction error. A real deadline guarantee requires admission control: the system must reject, defer or degrade some work when guaranteed demand exceeds capacity.

Priority scheduling decides who suffers under contention. Admission control determines whether the promised service level is feasible at all.

16. Decision pseudocode and implementation checklist

16.1 Simplified one-worker scheduler pseudocode

```

while server_is_running:
    ingest_tokenized_requests_and_control_messages()

    for each incoming request:
        validate_request()
        build_internal_Req()
        if grammar_not_ready:
            grammar_queue.add(req)
        else:
            validate_priority_and_queue_limit()
            maybe_start_storage_KV_prefetch()
            waiting_queue.append(req)

    process_previous_batch_result()
    merge_completed_prefills_into_running_batch()
    remove_finished_requests()
    move_grammar_ready_requests_to_waiting_queue()
    update_cache_prefetch_events()

    reorder(waiting_queue, selected_policy)

    prefill_batch = []
    initialize_admission_budgets_from:
        free_and_evictable_KV
        request_slots
        running_decode_reservations
        prefill_and_chunk_limits

    for req in waiting_queue_order:
        compute_prefix_match_and_uncached_length(req)
        if cache_restore_pending or incompatible_LoRA:
            continue
        if req_fits_all_budgets:
            admit_fully_or_as_chunk(req)
        elif priority_preemption_can_create_space:
            preempt_lower_priority_work_and_retry(req)
        else:
            stop_or_skip_according_to_the_failed_constraint

    if prefill_batch_is_not_empty:
        next_batch = prefill_batch
        optionally_mix_running_decode_requests()
    else:
        filter_finished_running_requests()
        if decode_KV_is_unsafe:
            retract_requests_and_increase_reservation_ratio()
        next_batch = running_decode_batch

    run_model_forward(next_batch)
    process_and_stream_results()

```

16.2 Facts your implementation should log

Area	Minimum observability
Queue decision	Request ID, priority, queue time, active policy, rank before/after and reason for ordering.
Cache decision	Prompt tokens, device/host/storage hits, uncached tokens, transfer state and recomputation avoided.
Admission decision	Available/evictable KV, reserved decode KV, request rows, prefill budget, requested charge and rejection reason.
Chunk decision	Original prompt remainder, chosen chunk size, time per chunk and urgent arrival delay.
Preemption/retraction	Victim request, priority gap, KV released, work lost/reprocessed and time until resumption.
Outcome	Queue delay, prefill start/end, first-token time, inter-token latency, deadline status and background throughput.

16.3 Practical implementation order

- Instrument the native scheduler before changing policy. Record queue order, prefix hits, exact admission failures, batch type and retractions.
- Start with priority plus FCFS and chunked prefill. Establish a clear baseline before combining cache-aware ranking with business priority.
- Add harness metadata at the boundary, but translate it into a small trusted internal schema instead of passing arbitrary frontend values directly into low-level controls.
- Implement deadline slack as a controller score or priority update, while preserving SGLang's admission checks.
- Add cache preparation as an independent action path so a session can be prefetched before its resumed model request arrives.
- Stress the system until deadlines fail, then attribute every miss to queueing, kernel boundary, KV restoration, prefill interference, lack of compute, or prediction error.
- Only after the best software design is measured should any fast path be considered for firmware or hardware acceleration.

16.4 Bottom line

SGLang already contains strong machinery for token-aware ranking, memory-safe admission, continuous batching, cache reuse, chunked prefill and recovery. The proposed controller should extend that machinery with agent intent and future-work knowledge, not replace it.

17. Primary sources

The analysis is based on official SGLang sources. GitHub main-branch line numbers can drift after new commits; symbol and file names are included so the relevant logic remains findable.

[1] Scheduler implementation

`sglang/python/sglang/srt/managers/scheduler.py`

<https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/scheduler.py>

Event loop, queue insertion, priority validation, queue limits, prefill-vs-decode selection, PrefillAdder invocation, batch construction, running-batch updates and retraction requeueing.

[2] Scheduling policy and PrefillAdder

`sglang/python/sglang/srt/managers/schedule_policy.py`

https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/schedule_policy.py

FCFS, LPM, DFS-weight, HRRN, LOF, random and routing-key ordering; prefix matching; in-batch prefix caching; token/KV admission budgets; priority preemption.

[3] Request and batch structures

`sglang/python/sglang/srt/managers/schedule_batch.py`

https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/schedule_batch.py

Req fields, KV ownership metadata, ScheduleBatch lifecycle, decode-memory checks and request retraction.

[4] Model forward batch

`sglang/python/sglang/srt/model_executor/forward_batch_info.py`

https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/model_executor/forward_batch_info.py

Conversion from CPU-side scheduling information to low-level GPU tensor inputs.

[5] Server arguments

SGLang server-argument documentation

https://docs.sglang.io/advanced_features/server_arguments.html

Current schedule-policy default and choices, priority flags, preemption threshold, queue limits, prefill tokens, chunking and related configuration.

[6] SGLang paper

SGLang: Efficient Execution of Structured Language Model Programs

<https://arxiv.org/abs/2312.07104>

RadixAttention, cache-aware scheduling, continuous batching, KV reuse motivation and original pseudocode.

[7] SGLang-JAX scheduler architecture

`docs/architecture/03-scheduler.md`

<https://github.com/sgl-project/sglang-jax/blob/main/docs/architecture/03-scheduler.md>

A readable official architecture narrative for analogous scheduler concepts. It is supplementary and should not be mistaken for the Python SRT implementation.

Source-to-topic map

Topic	Primary source
Runtime states and event loop	[1] Scheduler implementation
Request construction and native metadata	[1] Scheduler implementation; [3] Request and batch structures
Ranking policies	[2] Scheduling policy and PrefillAdder
Prefix matching and cache-aware scheduling	[2] Scheduling policy; [6] SGLang paper
Admission budgets	[2] Scheduling policy and PrefillAdder
Prefill/decode batch formation	[1] Scheduler implementation; [3] Request and batch structures
Decode memory and retraction	[1] Scheduler implementation; [3] Request and batch structures
Configuration flags and defaults	[5] Server arguments

Research conclusion: the native scheduler has detailed computational knowledge of requests and live GPU/cache state, but agentic intent, future tool timing and deadline semantics must be supplied by an external harness-aware controller.